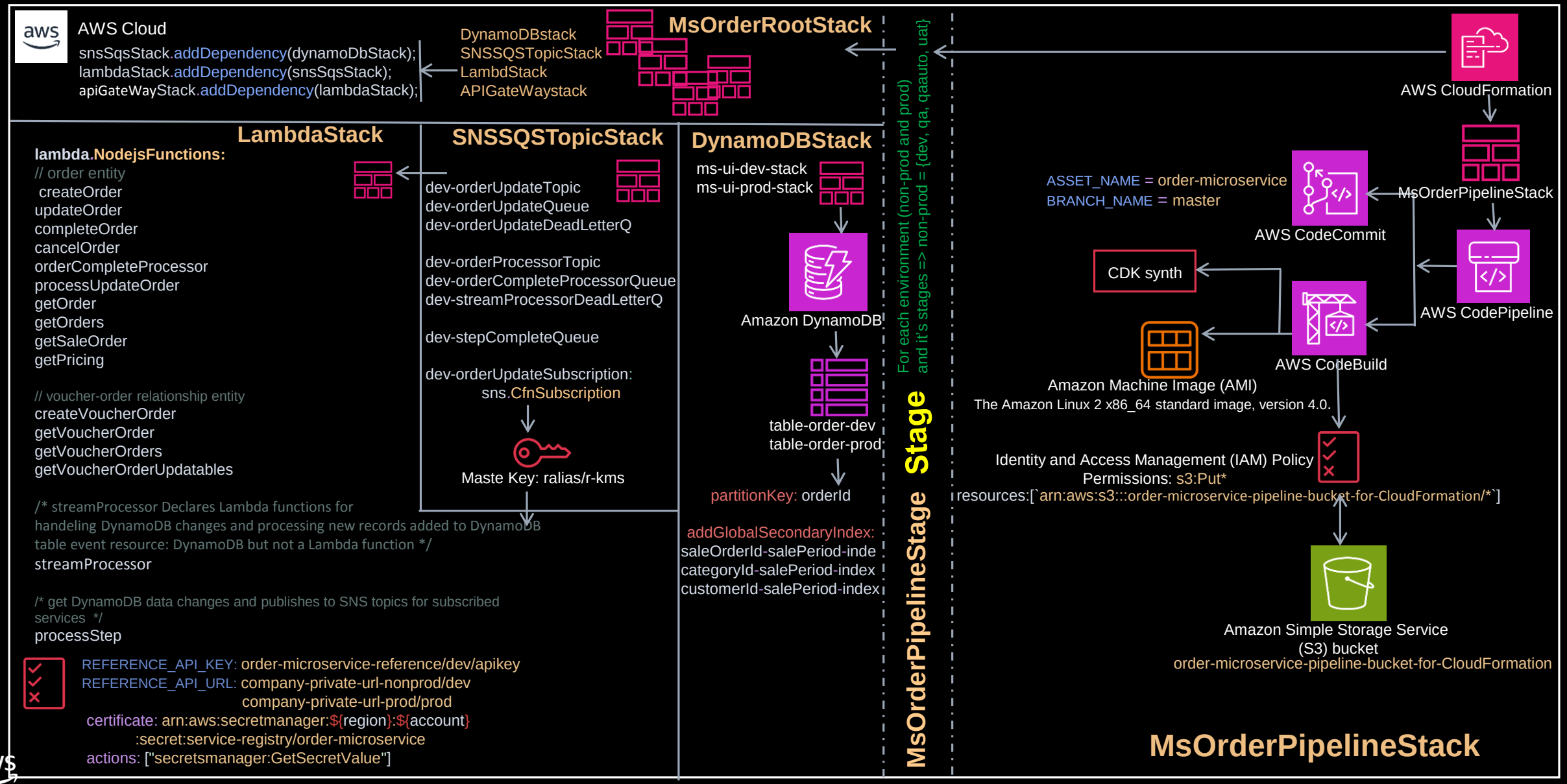
Max Montaseri

AWS Infrastructure as Code

Serverless Microservice (Back end)

aws

Stage: An abstract application modeling unit consisting of Stacks that should be deployed together. Derive a subclass of Stage and use it to model a single instance of your application. You can then instantiate your subclass multiple times to model **multiple copies of your application** which should **be deployed to different environments**.



AWS Cloud

```
// Set DynamoDB Policies
const dynamoOrderTableGetPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["dynamodb:Query", "dynamodb:GetItem", "dynamodb:BatchGetItem"],
  resources: [
    orderTable.tableArn,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_SALE_ORDERID}`,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CATEGORYID}`,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CUSTOMERID}`,
  ],
});

const dynamoOrderTableUpdatePolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["dynamodb:GetItem", "dynamodb:UpdateItem", "dynamodb:PutItem"],
  resources: [orderTable.tableArn],
});

const KmsKeyPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: [
    "kms:DescribeKey",
    "kms:GenerateDataKey",
    "kms:Encrypt",
    "kms:ReEncrypt*",
    "kms:Decrypt",
  ],
  resources: [KmsKey.keyArn],
});


const secretsmanagerPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  resources: [secretApiKeyArn + "-*"],
  actions: ["secretsmanager:GetSecretValue"],
});
```

LambdaStack

```
const lambdaProps = {
    runtime: Runtime.NODEJS_16_X,
    timeout: Duration.seconds(LAMBDA_DEFAULT_TIMEOUT),
    logRetention: RetentionDays.THREE_MONTHS,
    tracing: Tracing.ACTIVE,
    memorySize: 1024,
    bundling: {
      environment: { NODE_ENV: "production" },
      target: "node16",
      keepNames: true,
      sourceMap: true,
     },
    environment: {
      ENV_NAME: `${stackEnv}`,
      LOG_LEVEL: `${logLevel}`,
      TABLE_ORDER: orderTable.tableName,
      ORDER_PROCESSOR_TOPIC: orderProcessorTopic.topicArn,
      ORDER_UPDATE_TOPIC: orderUpdateTopic.topicArn,
      REFERENCE_API_KEY,
      REFERENCE_API_URL,
      PRIVATE_APIGW_BASE_URL: `${vpcEndpoint}/${stackEnv}`,
      SALE_API_ID: `${voucherApiId}`,
      REGISTRATION_API_ID: `${registrationApiId}`,
      CUSTOMER_API_ID: `${customerApiId}`,
   },
   };

REFERENCE_API_KEY: order-microservice-reference/dev/apikey
REFERENCE_API_URL: company-private-url-nonprod/dev
```

types.ts

```
import { StackProps, StageProps, Duration, NestedStackProps, Environment,} from "aws-cdk-lib";
import * as lambda from "aws-cdk-lib/aws-lambda-nodejs";
import * as dynamodb from "aws-cdk-lib/aws-dynamodb";
import * as sqs from "aws-cdk-lib/aws-sqs";
import * as sns from "aws-cdk-lib/aws-sns";
```

MsOrderPipelineStackProps

```
vpcId: string;
waves: Array<MsOrderPipelineStageProps>;
```

```
name:string;
stageName: string;
tags?:{};
approval?: boolean;
logLevel?: string;
environments: Array<MsOrderStackProps>;
```

```
stackEnv: string;
stackEnvStage: string;
logLevel?: string;
pitr?: boolean;
maxBatchingWindow?: Duration;
voucherApiId?: string;
registrationApiId?: string;
customerApiId?: string;
categoryApiId?: string;
```

types.ts

```
interface BaseProps extends NestedStackProps {

  stackEnv: string;
  logLevel?: string;
  pitr?: boolean;
  maxBatchingWindow?: Duration;
  voucherApiId?: string;
  registrationApiId?: string;
  customerApiId?: string;
  categoryApiId?: string;

}
```

types.ts

```
interface ApiGWStackProps extends BaseProps {

  codeBaseEnv: string;
  streamProcessor: lambda.NodejsFunction;

  createOrder: lambda.NodejsFunction;
  updateOrder: lambda.NodejsFunction;
  completeOrder: lambda.NodejsFunction;
  cancelOrder: lambda.NodejsFunction;

  orderCompleteProcessor: lambda.NodejsFunction;
  processStep: lambda.NodejsFunction;

  getOrder: lambda.NodejsFunction;
  getOrders: lambda.NodejsFunction;
  getSaleOrder: lambda.NodejsFunction;
  getPricing: lambda.NodejsFunction;

  createVoucherOrder: lambda.NodejsFunction;

  getVoucherOrder: lambda.NodejsFunction;
  getVoucherOrders: lambda.NodejsFunction;
  getVoucherOrderUpdatables: lambda.NodejsFunction;

}
```

```
interface LambdaStackProps extends BaseProps {

  codeBaseEnv: string;
  orderTable: dynamodb.Table;

  orderUpdateTopic: sns.Topic;
  orderUpdateQueue: sqs.Queue;
  orderUpdateDeadLetterQ: sqs.Queue;

  orderProcessorTopic: sns.Topic;
  orderCompleteProcessorQueue: sqs.Queue;
  streamProcessorDeadLetterQ: sqs.Queue;

  stepCompleteQueue: sqs.Queue;

 }
```

config.ts

```
const MS_NAME='order' // MicroserviceName (Exp. order)
const ENV_PROD = { account: '????7645????' , region: 'us-east-1' };
const PROD_VPCID = 'vpc-????d01986b04????'; //'vpc-prod' // already should be created in account
const PROD_SG = 'vpcLambda-sg-prod'; // created by cdk-infrastructure-pipeline
const APIGW_VPC_ENDPID_PROD = 'ApiVpcEndpoint-prod'; //'vpce-????88fe6afc9????';// created by cdk-infrastructure-pipeline
export const REFERENCE_API_URL_BASE_PROD = 'company-private-api-url-prod'; //?? will be used in LambdaStack: new lambda.NodejsFunction

const ENV_NONPROD = { account: '????7645????' , region: 'us-east-1' };
const NONPROD_VPCID = 'vpc- '????a22dbbd46????'; //'vpc-prod' // alreday should be created in account
const NONPROD_SG = 'vpcLambda-sg-nonprod'; // created by cdk-infrastructure-pipeline
const APIGW_VPC_ENDPID_NONPROD = 'ApiVpcEndpoint-nonprod'; //'vpce-????1b53c0a5????'; // created by cdk-infrastructure-pipeline
export const REFERENCE_API_URL_BASE_NON_PROD ='company-private-api-url-nonprod'; //?? will be used in LambdaStack: new lambda.NodejsFunction

export const IDENTITY_PROXY_API_URL = 'bsidentity/rest/identitys'; // will be used in LambdaStack: new lambda.NodejsFunction
export const REFERENCE_API_KEY_BASE = `${MS_NAME}-microservice-reference`;
//will be used in LambdaStack for Secrete Manager (SM): secretApiKeyArn = sm.Secret.fromSecretNameV2 and secretsmanagerPolicy configurations

export const DEPENDENCY_TRACK_S3 = `${MS_NAME}-microservice-pipeline-bucket-for-CloudFormation`;
// will be used in MsOrderPipelineStack: CodeBuildStep
// this Microservice Pipekine will save the MicroservicePipelineStack built artifacts (by CodePipekine) for executing by AWS Cloudformation

export const PIPELINE_NAME = `${MS_NAME.toUpperCase}-Microservice-Pipeline`;
export const BRANCH_NAME = 'master'; // will be used in MsOrderPipelineStack: CodeBuildStep

export const ASSET_NAME = `${MS_NAME}-microservice`;
/* will be used in MsOrderPipelineStage: stage name (id) and MsOrderPipelineStack: CodeBuildStep.projectName
and APIGatewayStack: lambdaApi = new LambdaRestApi(this, `${stackEnv}-${ASSET_NAME}` */

export const DB_TABLE_ORDER = `table-${MS_NAME.toLowerCase}`; // will be used in DynamoDbStack
```

config.ts

```
// DEPARTMENT=SALE => department=sale
export const DB_TABLE_ORDER_INDEX_SALE_ORDERID = 'saleOrderId-salePeriod-index';
export const DB_TABLE_ORDER_INDEX_CATEGORYID = 'categoryId-salePeriod-index';
export const DB_TABLE_ORDER_INDEX_CUSTOMERID = 'customerId-salePeriod-index';

export const KMS_ALIAS_NAME = `alias/${MS_NAME}-kms`; // created by cdk-infrastructure-pipeline
// 'alias/r-kms';
/* will be used in LambdaStack: find kms.Key and KmsKey.keyArn will be used in KmsKeyPolicy
and in SNSSQSTopicStack: as masterkey for creating sns.Topic and encryptionMasterKey in creating sqs.Queue */

// Used in SQS: visibilityTimeout: Duration.seconds(LAMBDA_DEFAULT_TIMEOUT * 6),
export const LAMBDA_DEFAULT_TIMEOUT = 30; // seconds;
/* will be used in LambdaStack as lambdaProps for new lambda.NodejsFunction
and in SNSSQSTopicStack: new sqs.Queue for visibilityTimeout */
export const LAMBDA_ORDER_PROCESSOR = `${MS_NAME.toLowerCase}-processor`;
/* will be used in LambdaStack as part of entry path name for new lambda.NodejsFunction
 entry: `src/${LAMBDA_ORDER_PROCESSOR}/create-order.js`,
=> cdk-microserviceorder-pipeline/src/order-processor/ => Javascript files */

export const SERVICE_REGISTRY_SECRET_KEY = `service-registry/${MS_NAME.toLowerCase}-microservice`;
/* will be used in LambdaStack as serviceRegistryProps in new lambda.NodejsFunction to detect the Lambda resgidters to
which environment Emp. codeBaseEnv: 'devops-dev',
Also creating an IAM resource access permission in IAM policy: new iam.PolicyStatement
 => resources: `arn:aws:secretsmanager:${region}:${account}:secret:${SERVICE_REGISTRY_SECRET_KEY}-*`
               actions: ["secretsmanager:GetSecretValue"]
 */

export const PAYMENT_URL = ''; // Private payment URL
/* will be used in LambdaStack: this.createOrder = new lambda.NodejsFunction(this, "createorder", {
create-order.js => import { createOrder, getPaymentUrl } from '../services/order' */
```

aws

config.ts

```
export const getTags = (env: string) => ({
  asset_id: ASSET_NAME,
  system_id: `${MS_NAME}`,
  accessibility: 'private',
  data_classification: 'proprietary',
  env_intent: env,
  engineer: 'backend@company.org',
});

export const getEnvironmentConfig = (env: string, codeBaseEnv: string) =>
/* LambdaStack and APIGatewayStack will invoke getEnvironmentConfig(stackEnv, codeBaseEnv) => Exp. codeBaseEnv:'devops-dev'
     LambdaStack: for initialling identityProxyApiUrl and other infrustructure properties for prod/nonprod
     APIGatewayStack: for creating LambdaRestApi and it's iam.PolicyDocument  */
env === 'prod'
    ? {
        apigw_vpcEndpoint: APIGW_VPC_ENDPID_PROD,
        vpcId: PROD_VPCID,
        sgName: PROD_SG,
        referenceApiBaseUrl: REFERENCE_API_URL_BASE_PROD,
        vpcEndpoint: `${APIGW_VPC_ENDPID_PROD}-????.execute-api.us-east-1.vpce.amazonaws.com`,
        // or 'https://vpce-????88fe6afc9????-????.execute-api.us-east-1.vpce.amazonaws.com',
        identityProxyApiUrl: IDENTITY_PROXY_API_URL,
        identityProxyEnv: 'prod',
      }
    : {
        apigw_vpcEndpoint: APIGW_VPC_ENDPID_NONPROD,
        vpcId: NONPROD_VPCID,
        sgName: NONPROD_SG,
        referenceApiBaseUrl: REFERENCE_API_URL_BASE_NON_PROD,
        vpcEndpoint: `${APIGW_VPC_ENDPID_NONPROD}-????.execute-api.us-east-1.vpce.amazonaws.com`,
        // or 'https://vpce-????e1b53c0a????-????.execute-api.us-east-1.vpce.amazonaws.com',
        identityProxyApiUrl: `${codeBaseEnv}/${IDENTITY_PROXY_API_URL}`,
        identityProxyEnv: 'preprod', // preprod <=> Dev, Systest, QA, UAT, ...
      };
```

config.ts

```
export const PIPELINE_CONFIG = {
  env: ENV_NONPROD,
  tags: getTags('nonprod'),
  vpcId: NONPROD_VPCID,
  waves: [
    {
      env: ENV_NONPROD,
      name: 'dev-stage',
      stageName: 'dev-stage',
      tags: getTags('dev'),
      approval: false,// doesn't need to be approved for dev environment in Pipeline
      logLevel: 'debug',
      environments: [
      // * types => MsOrderPipelineStageProps
        {
          // *other props from StackProps
          env : ENV_NONPROD,
          tags: getTags('dev'),
          codeBaseEnv: 'devops-dev',
          description : 'This is dev stage/env for developer testing',

          // types => MsOrderStackProps
          stackEnv: 'dev',
          stackEnvStage: 'dev-stage',
          // logLevel?: string,
          // pitr?: boolean;
          // maxBatchingWindow?: Duration,
          voucherApiId: '1zm4g6u????',
          registrationApiId:'??????',
          customerApiId: '99a6z????',
          categoryApiId: 'o9t5sk????',
        },
      ]
    },
```

aws

config.ts

```
    {
        env: ENV_NONPROD,
        name: 'qa-stage',
        stageName: 'qa-stage',
        tags: getTags('qa'),
        approval: true,
        logLevel: 'debug',
        environments: [
          {
            // *other props from StackProps
            env : ENV_NONPROD,
            tags: getTags('qa'),
            codeBaseEnv: 'devops-qa',
            description : 'This is qa stage/env for deveoper testing',

            stackEnv: 'qa',
            stackEnvStage: 'qa-stage',
            // logLevel?: string,
            // pitr?: boolean;
            // maxBatchingWindow?: Duration,
            voucherApiId: '2zm4g6u????',
            registrationApiId: '??????',
            customerApiId: '99a6zj????',
            categoryApiId: 'o9t5sk????',
          },
        ]
      },
```

aws

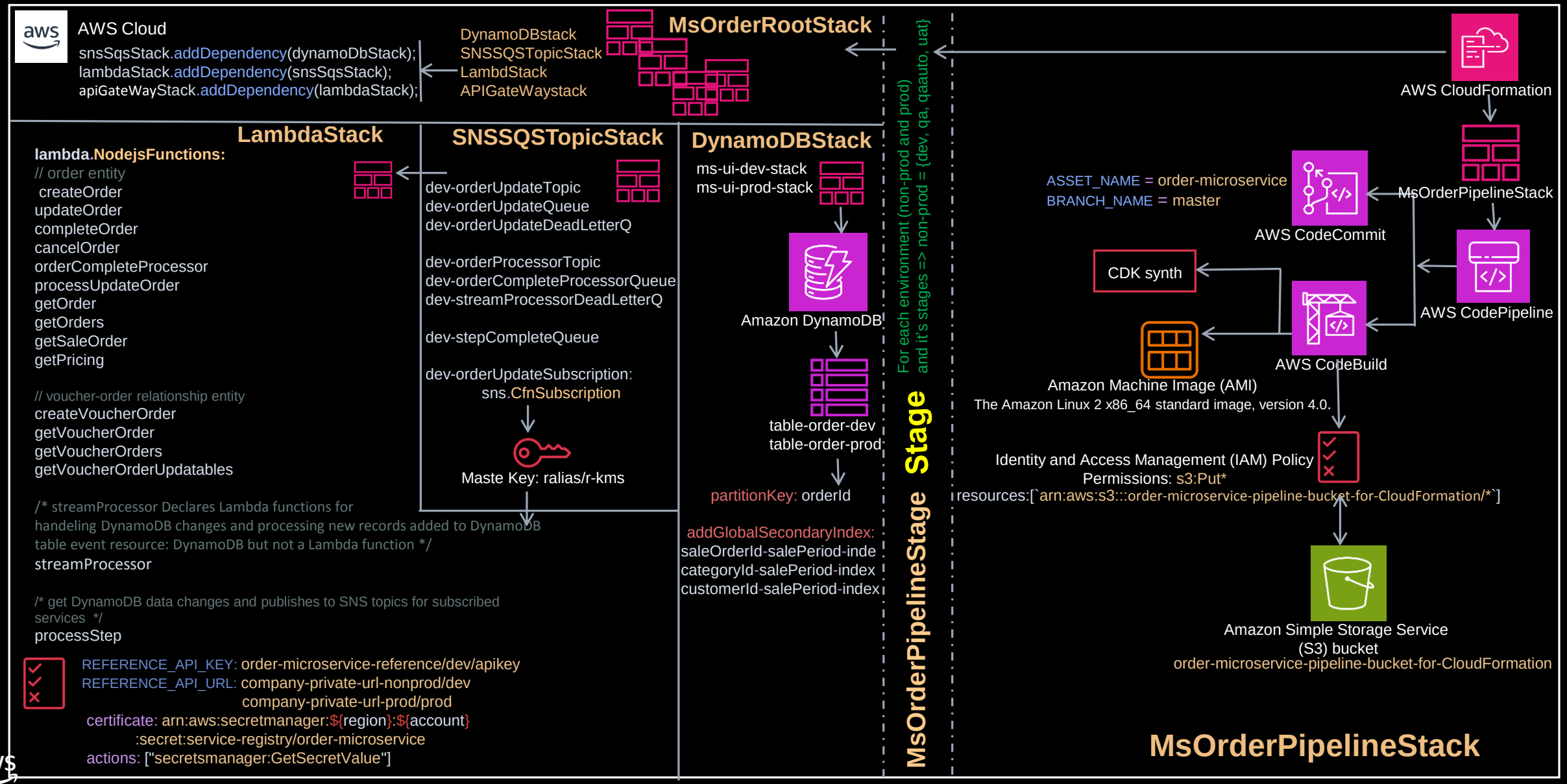
config.ts

```
    {
      env: ENV_PROD, // Production
      name: 'prod-stage',
      stageName: 'prod-stage',
      tags: getTags('prod'),
      approval: true,
      logLevel: 'info',
      environments: [
        {
          // *other props from StackProps
          env : ENV_PROD,
          tags: getTags('prod'),
          codeBaseEnv: 'devops-prod',
          description : 'This is prod stage/env for end user',

          stackEnv: 'prod',
          stackEnvStage: 'prod-stage',
          pitr: true,
        }
      ]
    },
  ],
};
```

aws

Stage: An abstract application modeling unit consisting of Stacks that should be deployed together. Derive a subclass of Stage and use it to model a single instance of your application. You can then instantiate your subclass multiple times to model **multiple copies of your application** which should **be deployed to different environments**.



AWS Cloud

```
// Set DynamoDB Policies
const dynamoOrderTableGetPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["dynamodb:Query", "dynamodb:GetItem", "dynamodb:BatchGetItem"],
  resources: [
    orderTable.tableArn,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_SALE_ORDERID}`,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CATEGORYID}`,
    `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CUSTOMERID}`,
  ],
});

const dynamoOrderTableUpdatePolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: ["dynamodb:GetItem", "dynamodb:UpdateItem", "dynamodb:PutItem"],
  resources: [orderTable.tableArn],
});

const KmsKeyPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  actions: [
    "kms:DescribeKey",
    "kms:GenerateDataKey",
    "kms:Encrypt",
    "kms:ReEncrypt*",
    "kms:Decrypt",
  ],
  resources: [KmsKey.keyArn],
});


const secretsmanagerPolicy = new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  resources: [secretApiKeyArn + "-*"],
  actions: ["secretsmanager:GetSecretValue"],
});
```

LambdaStack

```
const lambdaProps = {
    runtime: Runtime.NODEJS_16_X,
    timeout: Duration.seconds(LAMBDA_DEFAULT_TIMEOUT),
    logRetention: RetentionDays.THREE_MONTHS,
    tracing: Tracing.ACTIVE,
    memorySize: 1024,
    bundling: {
      environment: { NODE_ENV: "production" },
      target: "node16",
      keepNames: true,
      sourceMap: true,
     },
    environment: {
      ENV_NAME: `${stackEnv}`,
      LOG_LEVEL: `${logLevel}`,
      TABLE_ORDER: orderTable.tableName,
      ORDER_PROCESSOR_TOPIC: orderProcessorTopic.topicArn,
      ORDER_UPDATE_TOPIC: orderUpdateTopic.topicArn,
      REFERENCE_API_KEY,
      REFERENCE_API_URL,
      PRIVATE_APIGW_BASE_URL: `${vpcEndpoint}/${stackEnv}`,
      SALE_API_ID: `${voucherApiId}`,
      REGISTRATION_API_ID: `${registrationApiId}`,
      CUSTOMER_API_ID: `${customerApiId}`,
   },
   };

REFERENCE_API_KEY: order-microservice-reference/dev/apikey
REFERENCE_API_URL: company-private-url-nonprod/dev
```

MsOrderPipelineStack (CodeBuildStep, CodePipelineSource.codeCommit and CodePipeline)

will be executed whenever `PIPELINE_CONFIG` **or** `config.ts` **configurations are changed in CodeCommit repository connected to** *ASSET_NAME, BRANCH_NAME* **and** *DEPENDENCY_TRACK_S3*. Built artifacts saved/put in DEPENDENCY_TRACK_S3 S3 Bucket will be put in each stage's (prod/nonprod) VPC.
Then reads the pipeline's stages (waves) and their environments (StackEnv, VPC) from `PIPELINE_CONFIG.waves`**. Finally invokes the** `MsOrderRootStack` **for provisioning AWS resources DynamoDbStack, SNSSQSTopicStack, LambdaStack and apigwStack in each stackEnv, VPC and** *ASSET_NAME (Project or Mircoservice)*

```
const app = new cdk.App(); new MsOrderPipelineStack(app, PIPELINE_NAME, PIPELINE_CONFIG);
```

MsOrderPipelineStack

```
const { waves, vpcId } = props || {};
    const vpc = Vpc.fromLookup(this, 'vpc', { vpcId })
    const pipeline = new CodePipeline(this, "Pipeline", {
      selfMutation: true,
      pipelineName: PIPELINE_NAME,
      crossAccountKeys: true,
      codeBuildDefaults: {
        // CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-msName
        // and doesn't exist in InfrastructurePipelineStack
        buildEnvironment: {
          // see https://docs.aws.amazon.com/codebuild/latest/userguide/build-env-ref-available.html
          buildImage: LinuxBuildImage.AMAZON_LINUX_2_4,
        },
      },
      synth: new CodeBuildStep("Synth", {
      // Synth => will take our codes then generate AWS CloudFormation templates
      // => we can see the generated templates=> this happens always we deploy!
        projectName: ASSET_NAME,
        input: CodePipelineSource.codeCommit(
          Repository.fromRepositoryName(this, "repo", ASSET_NAME),
          BRANCH_NAME
        ),
        commands: [
          "node -v",
          "npm ci",
```

aws

MsOrderPipelineStack

```
        "npm test",
        "npm run build",
        "npx cdk synth",
      ],
      rolePolicyStatements: [
        // CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-backend
        // and doesn't exist in InfrastructurePipelineStack
        new PolicyStatement({
          effect: Effect.ALLOW,
          actions: ["s3:Put*"],
          // Built artifacts will be saved/put in S3 Bucket: DEPENDENCY_TRACK_S3
          resources: [`arn:aws:s3:::${DEPENDENCY_TRACK_S3}/*`],
        }),
      ],
      partialBuildSpec: BuildSpec.fromObject({
        // CodeBuild & Unit testing stage (BuildSpec) only used in UIPipelineStack and cdk-microservice-backend
        // and doesn't exist in InfrastructurePipelineStack
        reports: {
          coverageReports: {
            files: "clover.xml",
            "file-format": "CLOVERXML",
            "base-directory": "coverage",
          },
          jestReports: {
            files: "unitTestReport.xml",
            "file-format": "JUNITXML",
            "base-directory": "coverage",
          },
        },
      }),
      // Built artifacts saved/put in DEPENDENCY_TRACK_S3 S3 Bucket will be put in each stage's (prod/nonprod) VPC
      vpc,
      // vpc: Vpc.fromLookup(this, "vpc", { vpcId }),
    }),
  });
```



MsOrderPipelineStack

```
      waves.forEach((w) => {
        const { environments, approval, stageName = "", ...common } = w;
        // Add stages (dev-stage, test-stage, qa-stage, ..., prod-stage)
        // and their manual deploy approval to CodePipeline wave
        const wave = pipeline.addWave(stageName, {
          pre: approval
            ? [new ManualApprovalStep(`promoteto${stageName}`)]
            : undefined,
        });

        environments.forEach((e) => {
          wave.addStage(
            // for all defined pipeline stages (stageName) in waves and each stage's environmnts (stackEnv)
            // PipelineStage would invoke the RootStack for provisioning AWS resources DynamoDbStack, SNSSQSTopicStack,
            // LambdaStack and apigwStack
            new MsOrderPipelineStage(this, `${ASSET_NAME}-${e.stackEnv}`, {...common, ...e, })
          );
        });
      });
    }
  }
```

aws

MsOrderPipelineStage invoked by **MsOrderPipelineStack** for each pipeline stage. then invokes **MsOrderRootStack** for provisioning AWS resources DynamoDbStack, SNSSQSTopicStack, LambdaStack and apigwStack in each stackEnv regarding each environment (stackEnv, VPC and *ASSET_NAME (Project or Mircoservice)*)

```
new MsOrderRootStack(this, "msorder-stack", props);
```

MsOrderRootStack provisions AWS services as cloud infrastructure for each stage (prod, nonprod) and it's environments (Dev, SysTest, UAT, DR, Sandbox)

Why AWS CloudFormation nested stack? in each AWS account there is limitation for creating resources per each CloudFormation stack

- **RootStack:** for make child stack dependent to each other. RootStack will create it's infrastructure child (nested) stacks: **DynamoDbStack, SNSSQSTopicStack, LambdaStack** and **APIGatewayStack** one by one, then pass their output to next child stack (abstraction and easy understanding)

DynamoDbStack output (DynamoDB Table) => pass to => SNSSQSTopicStack props => pass to => LambdaStack props => pass to => APIGateWayStack

- Finally addDependency method should be invoked for the nested stack creation order:

```
snsSqsStack.addDependency(dynamodbStack);
lambdaStack.addDependency(snsSqsStack);
apigwStack.addDependency(lambdaStack);
```

aws

MsOrderRootStack

```
const dynamodbStack = new DynamoDbStack(this, "dynamodb-stack", props);
const { orderTable } = dynamodbStack;
// AWS Integration & Messaging (Decoupling applications: SQS, SNS, Kinesis, Active MQ)
const snsSqsStack = new SNSSQSTopicStack(this, "sns-sqs-stack", props);

const {
  orderProcessorTopic,
  orderUpdateTopic,
  streamProcessorDeadLetterQ,
  orderUpdateDeadLetterQ,
  stepCompleteQueue,
  orderCompleteProcessorQueue,
  orderUpdateQueue,
} = snsSqsStack;

const lambdaStack = new LambdaStack(this, "lambda-stack", {
  // SNSSQSTopicStack props and DynamoDbStack output (as a DynamoDB Table) => pass to => LambdaStack
  ...props,
  codeBaseEnv: '', // Added for fix the error
  orderTable,
  orderProcessorTopic,
  orderUpdateTopic,
  streamProcessorDeadLetterQ,
  orderUpdateDeadLetterQ,
  stepCompleteQueue,
  orderCompleteProcessorQueue,
  orderUpdateQueue,
});
```

MsOrderRootStack

```
        createVoucherOrder,
        getVoucherOrders,
        getVoucherOrder,
        getVoucherOrderUpdatables,
      });

      snsSqsStack.addDependency(dynamodbStack);
      lambdaStack.addDependency(snsSqsStack);
      apigwStack.addDependency(lambdaStack);

    }
```



DynamoDbStack

will create a DynamoDB table for each environmnts (stackEnv) defined in pipeline stages (stageName)'s waves

```
    const orderTableName = `${stackEnv}-${DB_TABLE_ORDER}`;
```

DynamoDB table has pointInTimeRecovery=true only for prod, DR and perf environments (config.ts), also timeToLiveAttribute for all envs

DynamoDB table has GlobalSecondaryIndex as indexes (config.ts): organize items on partitionKey (index) then sort by sortKey (salePeriod)

```
  orderTable: dynamodb.Table;
   constructor(scope: Construct, id: string, props: BaseProps) {
     super(scope, id, props);
     const { stackEnv, pitr } = props;
     const orderTableName = `${DB_TABLE_ORDER}-${stackEnv}`;
     this.orderTable = new dynamodb.Table(this, "table", {
       tableName: orderTableName,
       partitionKey: {
         // Primary Key (PK)
         name: "orderId",
         type: dynamodb.AttributeType.STRING
       },
       billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
       stream: dynamodb.StreamViewType.NEW_AND_OLD_IMAGES,
       pointInTimeRecovery: pitr,
       timeToLiveAttribute: "ttl",
     });
```

SNSSQSTopicStack

AWS Integration & Messaging (Decoupling applications: SQS, SNS, Kinesis, Active MQ)

SNSSQSTopicStack will send these SNS topica, SQS queues and CfnSubscription (CloudFormation Subscription) to LambdaStack

```
orderProcessorTopic: sns.Topic
streamProcessorDeadLetterQ: sqs.Queue
orderUpdateDeadLetterQ: sqs.Queue
orderCompleteProcessorQueue: sqs.Queue
orderUpdateQueue: sqs.Queue
stepCompleteQueue: sqs.Queue
orderUpdateTopic: sns.Topic
orderUpdateSubscription: sns.CfnSubscription
```

A dead-letter queue (DLQ) is a special type of message queue that temporarily stores messages that a software system cannot process due to errors.

orderProcessorTopic has 2 SqsSubscription: orderCompleteProcessorQueue and stepCompleteQueue

1-orderCompleteProcessorQueue

-orderCompleteProcessorQueue has filters: ORDER_COMPLETE and NOT_STARTED => If a subscription doesn’t have a filter policy, it receives every message

-orderCompleteProcessorQueue has orderCompleteProcessorDeadLetterQ: used as deadLetterQueue in orderCompleteProcessorQueue

2-stepCompleteQueue

-stepCompleteQueue has filters: COMPLETED => If a subscription doesn’t have a filter policy, it receives every message

-stepCompleteQueue has stepCompleteDeadLetterQ: used as deadLetterQueue in stepCompleteQueue

SNSSQSTopicStack: orderUpdateTopic and orderUpdateSubscription

```
this.orderUpdateTopic = new sns.Topic(this, 'sns-order-update-topic', {
  displayName: `Order Update Topic - ${stackEnv}`,
  topicName: `${stackEnv}-update-orders`,
  masterKey: orderKmsKey,
})

this.orderUpdateDeadLetterQ = new sqs.Queue(this, `orderUpdateDeadLetterQ`, {
  queueName: `${stackEnv}-process-update-order-deadletterq`,
  encryption: sqs.QueueEncryption.KMS,
  encryptionMasterKey: orderKmsKey,
})

this.orderUpdateQueue = new sqs.Queue(this, 'orderUpdateQueue', {
  queueName: `${stackEnv}-process-update-order`,
  encryption: sqs.QueueEncryption.KMS,
  encryptionMasterKey: orderKmsKey,
  deadLetterQueue: {
    queue: this.orderUpdateDeadLetterQ,
    maxReceiveCount: 3,
  },
  retentionPeriod: Duration.days(14),
  visibilityTimeout: Duration.seconds(LAMBDA_DEFAULT_TIMEOUT * 6),
})

this.orderUpdateQueue.addToResourcePolicy(
  new iam.PolicyStatement({
    effect: iam.Effect.ALLOW,
    actions: ['sqs:SendMessage'],
    resources: [this.orderUpdateQueue.queueArn],
    principals: [new iam.ServicePrincipal('sns.amazonaws.com')],
    conditions: {
      ArnEquals: { 'aws:SourceArn': this.orderUpdateTopic.topicArn },
    },
  })
)
```

aws

LambdaStack

```
const {
  stackEnv,
  codeBaseEnv,
  logLevel = "info",
  maxBatchingWindow = Duration.seconds(0),
  orderTable,
  orderProcessorTopic,
  orderUpdateTopic,
  streamProcessorDeadLetterQ,
  orderUpdateDeadLetterQ,
  stepCompleteQueue,
  orderUpdateQueue,
  orderCompleteProcessorQueue,
  voucherApiId,
  registrationApiId,
  customerApiId,
  categoryApiId,
} = props;
const { region, account } = Stack.of(this);
const {
/* LambdaStack and APIGatewayStack will invoke getEnvironmentConfig(stackEnv, codeBaseEnv) LambdaStack: for initialling
identityProxyApiUrl and other infrustructure properties for prod/nonprod
APIGatewayStack: for creating LambdaRestApi and it's iam.PolicyDocument  */
  vpcId,
  referenceApiBaseUrl,
  vpcEndpoint,
  sgName,
  identityProxyApiUrl,
  identityProxyEnv,
} = getEnvironmentConfig(stackEnv, codeBaseEnv);
const vpc = Vpc.fromLookup(this, "vpc-ms-order", { vpcId });
const vpcLambdaProps = {
  vpc,
  securityGroups: [
    SecurityGroup.fromLookupByName(this, "lambda-sg", sgName, vpc),
  ],
};
```



LambdaStack

Set Refrence API/URL, KMS, Secrets and Service Registery

```
const REFERENCE_API_KEY = `${REFERENCE_API_KEY_BASE}/${stackEnv}/apikey`;
const REFERENCE_API_URL = `${referenceApiBaseUrl}/${stackEnv}`;
const KmsKey = kms.Key.fromLookup(this, "KMS", {
  aliasName: KMS_ALIAS_NAME,
});
const secretApiKeyArn = sm.Secret.fromSecretNameV2(
  this,
  "secretApiKeyArn",
  REFERENCE_API_KEY
).secretArn;

const serviceRegistryProps = {
  environment: {
    SERVICE_REGISTRY_SECRET_KEY,
    CODE_BASE_ENV: codeBaseEnv,
  },
  policy: new iam.PolicyStatement({
    effect: iam.Effect.ALLOW,
    resources: [
      `arn:aws:secretsmanager:${region}:${account}:secret:${SERVICE_REGISTRY_SECRET_KEY}-*`,
    ],
    actions: ["secretsmanager:GetSecretValue"],
  }),
};
```

LambdaStack

```
    Set Lambda Props
        const lambdaProps = {
          runtime: Runtime.NODEJS_16_X,
          timeout: Duration.seconds(LAMBDA_DEFAULT_TIMEOUT),
          logRetention: RetentionDays.THREE_MONTHS,
          tracing: Tracing.ACTIVE,
          memorySize: 1024,
          bundling: {
            environment: { NODE_ENV: "production" },
            target: "node16",
            keepNames: true,
            sourceMap: true,
          },
          environment: {
            ENV_NAME: `${stackEnv}`,
            LOG_LEVEL: `${logLevel}`,
            TABLE_ORDER: orderTable.tableName,
            ORDER_PROCESSOR_TOPIC: orderProcessorTopic.topicArn,
            ORDER_UPDATE_TOPIC: orderUpdateTopic.topicArn,
            REFERENCE_API_KEY,
            REFERENCE_API_URL,
            PRIVATE_APIGW_BASE_URL: `${vpcEndpoint}/${stackEnv}`,
            SALE_API_ID: `${voucherApiId}`,
            REGISTRATION_API_ID: `${registrationApiId}`,
            CUSTOMER_API_ID: `${customerApiId}`,
          },
        };
```

LambdaStack

```
  Set DynamoDB Policies
      const dynamoOrderTableGetPolicy = new iam.PolicyStatement({
        effect: iam.Effect.ALLOW,
        actions: ["dynamodb:Query", "dynamodb:GetItem", "dynamodb:BatchGetItem"],
        resources: [
          orderTable.tableArn,
          `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_SALE_ORDERID}`,
          `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CATEGORYID}`,
          `${orderTable.tableArn}/index/${DB_TABLE_ORDER_INDEX_CUSTOMERID}`,
        ],
      });
      const dynamoOrderTableUpdatePolicy = new iam.PolicyStatement({
        effect: iam.Effect.ALLOW,
        actions: ["dynamodb:GetItem", "dynamodb:UpdateItem", "dynamodb:PutItem"],
        resources: [orderTable.tableArn],
      });

      const KmsKeyPolicy = new iam.PolicyStatement({
        effect: iam.Effect.ALLOW,
        actions: [
          "kms:DescribeKey",
          "kms:GenerateDataKey",
          "kms:Encrypt",
          "kms:ReEncrypt*",
          "kms:Decrypt",
        ],
        resources: [KmsKey.keyArn],
      });

      const secretsmanagerPolicy = new iam.PolicyStatement({
        effect: iam.Effect.ALLOW,
        resources: [secretApiKeyArn + "-*"],
        actions: ["secretsmanager:GetSecretValue"],
      });
```

aws

LambdaStack

DynamoDB Stream Processor Lambda function

streamProcessor Declares a Lambda function for handeling DynamoDB changes:

- processing changes/new records added to DynamoDB table
  event/trigger resource: DynamoEventSource but not a Lambda function

- onFailure: will created a SqsDlq for process deleting (streamProcessorDeadLetterQ)
  => new SqsDlq(streamProcessorDeadLetterQ)

also declate IAM policies for actions:

- SNS: Publish to orderUpdateTopic
- DynamoDB: GetRecords, DescribeStream, ListStreams and GetShardIterator from orderTable
- SNS: SendMessage to streamProcessorDeadLetterQ

LambdaStack

order microservice Lambda functions

- createOrder: event/trigger resource: APIGateWay

createOrder microservice Will take an order then create a record in DynamoDB orderTable using AWS KMS, AWS Secret Manager (ASM) and Service Registery policies also declate IAM policies for actions:

```
       - DynamoDB: PutItem */
```

- getOrders   event/trigger resource: APIGateWay returns a list of DynamoDB records from orderTable
- getOrder
- getOrders => getsaleOrder
- getPricing
- updateOrder
- completeOrder
- cancelOrder

aws

LambdaStack

processStep, processUpdateOrder and orderCompleteProcessor Lambda functions

```
const orderProcessorTopicPublishPolicy = new iam.PolicyStatement
/* order microservice processors will announce the changes  */

- processUpdateOrder = new lambda.NodejsFunction
/* event/trigger resource: APIGateWay */
/* processUpdateOrder will announce the changes through folllowing actions:
    - SNS: Publish to orderProcessorTopic
    - SQS: SendMessage to orderUpdateDeadLetterQ
    - SQS: ReceiveMessage to orderUpdateQueue

  processUpdateOrder uses:
    1- processStep to create an event for stepCompleteQueue
      - a SqsEventSource for stepCompleteQueue to reportBatchItemFailures

    2-processUpdateOrder to create an event for orderUpdateQueue
      - a SqsEventSource for orderUpdateQueue to reportBatchItemFailures
    */

- processStep = new lambda.NodejsFunction
/* processStep will get data changes from DynamoDB table
  and publishes to SNS topics for subscribed services  */

/*  processUpdateOrder uses:
  1- processStep to create an event for stepCompleteQueue
    - a SqsEventSource for stepCompleteQueue to reportBatchItemFailures   */
```

aws

APIGatewayStack

Users login => AWS Cognito

Users => Static content from => CloudFront distributaion => Origin Access from => S3 bucket

Users => invoke => APIGateWay triggers => proxy (false) => Lambda manipulate => SNS topics & SQS suscribers => DynamoDB

Users call REST API in APIGateWay, Then APIGateWay will trigger a Lambda function and Lambda manipulate the DynamoDB

```
  const {
    stackEnv,
    getOrders,
    getOrder,
    createOrder,
    updateOrder,
    cancelOrder,
    completeOrder,
    getSaleOrder,
    getPricing,
    createVoucherOrder,
    getVoucherOrders,
    getVoucherOrder,
    getVoucherOrderUpdatables,
    codeBaseEnv,
  } = props;
```

Infrastructure & VPC & apigw_vpcEndpoint

LambdaStack and APIGatewayStack will invoke getEnvironmentConfig(stackEnv, codeBaseEnv)
LambdaStack: for initialling identityProxyApiUrl and other infrustructure properties for prod/nonprod
APIGatewayStack: for creating LambdaRestApi and it's iam.PolicyDocument

```
  const { apigw_vpcEndpoint } = getEnvironmentConfig(stackEnv, codeBaseEnv);
```

aws